Spring 2026

CS 5757: Optimization Methods for Robotics
Homework 4: Sampling MPC and State Estimation

Due: Thursday, April 23, 2026, at 11:59pm on Gradescope, Total Points: 50

Learning goals for this problem set:

- **Problem 1:** Implement and build intution for how design choices (cost, action dimension) affect the performance of sampling-based MPC.
- **Problem 2:** Show and understand the connections between Gauss-Newton-based MAP inference and the Extended Kalman Filter.
- **Problem 3:** Implement factor graph-based SLAM and investigate how design choices like sparsity exploitation, robust losses impact performance and accuracy.

Submission Instructions: Submit your writeup as a PDF to the *Written Assignment* on Gradescope. For the *Programming Assignment*, submit *only* your completed `solutions.py`.
You are **strongly encouraged** to typeset your homework submissions using LaTeX(if you do not know LaTeXyet, now is as good a time as any to learn it -- you will need it in grad school anyway). If you opt for handwritten submissions, make sure that they are legible -- if we cannot read it, we cannot grade it!

4.1 Sampling-Based Model Predictive (15 pts)

Consider the **cheetah run** task from the DeepMind Control Suite, simulated in Mujoco. Figure 1 shows an overview of the robot.

Figure 1: Rendering of the half-cheetah robot from the DeepMind Control Suite

The cheetah has action dimension $d_u = 6$. At each real timestep the controller solves for an H-step action sequence by sampling, evaluating rollouts in parallel, and updating a search distribution.
To reduce the search dimension, we can optionally use a spline to parameterize the actions with a lower-dimensional curve, as in Mujoco MPC [1]. Algorithm 1 provides pseudocode for the algorithm.

Algorithm 1 Sampling-Based MPC

Require: Distribution parameters $\boldsymbol{\phi}$, horizon H, knot count n, sample count K

```
for each real timestep do
    Draw K samples {z^(k)}_{k=1}^K from q(z; φ)
    Expand each z^(k) via zero-order spline to H steps
    Roll out dynamics to obtain costs {J^(k)}_{k=1}^K
    Compute weights w_k = W(J^(1), ..., J^(K)) and update φ
    Apply the first knot's action, shift φ forward by one knot
end for
```

In this problem, we will consider three specific algorithms that implement this loop. All three maintain a mean $\boldsymbol{\mu} \in \mathbb{R}^{n \times d_u}$; CEM additionally maintains a per-element standard deviation $\boldsymbol{\sigma}$.

- Model-Predictive Path Integral Control (MPPI) computes baseline-subtracted softmax weights and sets the mean to the weighted average of all samples:

$$w_k = \frac{\exp\big(-(J^{(k)} - \min_j J^{(j)})/\lambda\big)}{\sum_{k'} \exp\big(-(J^{(k')} - \min_j J^{(j)})/\lambda\big)}, \qquad \boldsymbol{\mu} \leftarrow \sum_{k=1}^{K} w_k \, \boldsymbol{z}^{(k)}.$$

- The Cross-Entropy Method (CEM) selects the top-p fraction of samples by cost as the elite set $\mathcal{E}$ and refits the distribution to these elites:

$$\boldsymbol{\mu} \leftarrow \frac{1}{|\mathcal{E}|} \sum_{k \in \mathcal{E}} \boldsymbol{z}^{(k)}, \qquad \boldsymbol{\sigma} \leftarrow \mathrm{clip}\left(\sqrt{\frac{1}{|\mathcal{E}|} \sum_{k \in \mathcal{E}} (\boldsymbol{z}^{(k)} - \boldsymbol{\mu})^2}, \ \sigma_{\min}, \ \sigma_{\max} \right).$$

- Predictive Sampling (PS) simply sets the mean to the best sample: $\boldsymbol{\mu} \leftarrow \boldsymbol{z}^{(k^*)}$, where $k^* = \arg\min_k J^{(k)}$.

(a) (4 pts) Complete `mppi_update`, `cem_update`, and `ps_update` in `solutions.py`, implementing the three update rules described above. Carefully review all of the code in `sampling_mpc.py` before you continue.

Each method takes as inputs the knot points, `z_samples`, shape `(K, n, d_u)`, and the rollout costs`costs`, shape `(K,)`. `mppi_update` takes a keyword argument `lam`, the MPPI temperature λ, a `float`, and `cem_update` takes `elite_frac`, which defines the cost quantile corresponding to the elite set $\mathcal{E}$, as well as `sigma_min`, `sigma_max`, which are floats corresponding to the clipping values $\sigma_\min, \sigma_{\max}$.

(b) (3 pts) The optimization variable in Algorithm 1 is an H-step action sequence, so each sample lives in $\mathbb{R}^{H \cdot d_u}$. For the cheetah ($d_u = 6$, $H = 64$) this is $\mathbb{R}^{384}$. We reduce the search space by parameterizing the action sequence with $n \ll H$ knot points connected by a zero-order hold (ZOH) spline, so that the optimization variable is $\boldsymbol{z} \in \mathbb{R}^{n \cdot d_u}$.

A ZOH spline with sorted knot times $(t_0, \ldots, t_{n-1})$ and values $(z_0, \ldots, z_{n-1})$ evaluates at query time τ as $\mathrm{ZOH}(\tau) = z_{k(\tau)}$, where $k(\tau) = \max\{i : t_i \leq \tau\}$, clamped to $[0, n-1]$.

Complete the following functions in `solutions.py`:

- `query_zoh(knot_times, knot_values, query_times)` -– evaluate the ZOH spline at a

batch of query times. Takes sorted knot times `(n,)`, knot values `(n, d_u)`, and query times `(T,)`; returns the spline values, shape `(T, d_u)`.

- `shift_warmstart(knot_times, knot_values, ctrl_steps, fill_value)` -– after applying `ctrl_steps` simulator steps, the planning horizon advances. The warm-start knot values for the next MPC iteration are

$$z_i^{\text{new}} = \begin{cases} \text{ZOH}(t_i + \texttt{ctrl_steps}) & \text{if } t_i + \texttt{ctrl_steps} \leq t_{n-1}, \\ z_{\text{fill}} & \text{otherwise.} \end{cases}$$

  This function takes `knot_times`, shape `(n,)`, `knot_values`, shape `(n, d_u)`, `ctrl_steps`, the number of simulator steps applied since the last plan, and the `fill_value` z_{fill}, a `float` and returns the new knot points, shape `(n, d_u)`.

You can now run the planner interactively by running `python sampling_mpc.py --visualize`. Try it out! You can change different hyperparameters, switch between the updates, etc, to get a better feel for how they affect the overall performance.

After doing this, we'll discuss a few conceptual aspects of the algorithms.

(c) (2 pts) Run `python sampling_mpc.py --sweep spline`. This runs MPPI on cheetah with $K = 256$, $H = 64$, and $n \in \{1, 2, 4, 8, 16, 32, 64\}$ over $M = 10$ seeds (search spaces $\mathbb{R}^{24}$ through $\mathbb{R}^{384}$). Include the plot in your writeup. Increasing n makes the action sequence more expressive but also increases the dimension of the search space. Explain why there is a peak in the curve (i.e., why both $n = 1$ and $n = 64$ are sub-optimal).

(d) (2 pts) Run `python sampling_mpc.py --sweep methods`. This runs all three methods on cheetah with $n = 8$, $H = 64$, and $K \in \{64, 256, 1024, 4096\}$ over $M = 10$ seeds. Include the plot. One method should perform slightly worse - provide a hypothesis for why this might be the case.

(e) (1 pt) The starter code scales the sampling noise across knots via a noise ramp, as proposed in DIAL-MPC [2]. Knot $i \in \{0, \ldots, n-1\}$ is sampled as

$$z_i \sim \mathcal{N}\big(\mu_i,\ (\alpha_i\, \sigma_i)^2\big), \qquad \alpha_i = \frac{\alpha_{\max}}{n}\big(i + 1\big),$$

so $\alpha_1 = \alpha_{\max}/n$ and $\alpha_{n-1} = \alpha_{\max}$. Run `python sampling_mpc.py --sweep sigma_ramp`. This runs MPPI on cheetah with $n = 8$, $H = 64$, $K = 256$, and $\alpha_{\max} \in \{1.0, 1.5, 2.0, 3.0, 5.0\}$ over $M = 10$ seeds. Include the plot. Explain why increasing exploration at later knots is well-motivated by the structure of the MPC loop, and identify what goes wrong when $\alpha_{\max}$ is too large.

(f) (3 pts) Suppose you want to use sampling-based MPC to control a robot arm with a parallel-jaw gripper to pick a mug off a table. You have a MuJoCo model of the robot and a 3D scan of the mug. Your planner needs several things that were handed to you for free in parts (a)--(e). Answer the following questions on different practical aspects of the system:

- *State initialization.* At each MPC step you must set the initial state $\boldsymbol{s}_0$ of all K rollouts. What does this state include, what sensors provide it, and what happens if it is wrong?

- *Dynamics model.* Your rollouts use MuJoCo's contact dynamics. Name at least two specific ways the simulation could differ from reality that would cause the planned trajectory to fail on hardware.
- *Cost function.* In a few words/equations, describe a dense cost function that would enable the robot to pick up the mug and place it at a desired pose $\boldsymbol{T}_{\text{des}}$. Explain why a sparse cost like $\mathbf{1}[\text{mug not lifted}]$ might cause poor performance for a sampling-based algorithm.

Solution:

4.2 The Extended Kalman Filter as Single-Step Gauss-Newton (15 pts)

In class, we discussed a special case of the state estimation problem, where we have general nonlinear dynamics and measurements, but with additive Gaussian process and measurement noise:

$$\begin{aligned} \boldsymbol{x}_{t+1} &= \boldsymbol{f}(\boldsymbol{x}_t, \boldsymbol{u}_t) + \boldsymbol{w}_t, \quad \boldsymbol{w}_t \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{Q}), \\ \boldsymbol{y}_{t+1} &= \boldsymbol{h}(\boldsymbol{x}_{t+1}) + \boldsymbol{v}_{t+1}, \quad \boldsymbol{v}_{t+1} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{R}), \end{aligned}$$

where $\boldsymbol{w}_t$ is an additive, zero-mean Gaussian process noise with covariance $\boldsymbol{Q} \succ 0$, and $\boldsymbol{v}_{t+1}$ is an additive, zero-mean Gaussian measurement noise with covariance $\boldsymbol{R} \succ 0$.

A typical approach to state estimation for such systems is to use the Extended Kalman Filter (EKF). Assuming a Gaussian prior $p(\boldsymbol{x}_0)$, for a linear-Gaussian system, the Kalman Filter (KF) computes the exact Bayesian posterior $p(\boldsymbol{x}_t \mid \boldsymbol{u}_{0:t}, \boldsymbol{y}_{1:t}) = \mathcal{N}(\boldsymbol{\mu}_t, \boldsymbol{\Sigma}_t)$. For nonlinear systems, the key idea of the EKF is to apply an approximate KF update using linearizations of the dynamics and measurement functions at particular points:

$$\bar{\boldsymbol{\mu}} = \boldsymbol{f}(\boldsymbol{\mu}_t, \boldsymbol{u}_t), \tag{1}$$

$$\bar{\boldsymbol{\Sigma}} = \boldsymbol{Q} + \boldsymbol{A}_t \boldsymbol{\Sigma}_t \boldsymbol{A}_t^\top, \tag{2}$$

$$\boldsymbol{K} = \bar{\boldsymbol{\Sigma}} \boldsymbol{C}_{t+1}^\top (\boldsymbol{R} + \boldsymbol{C}_{t+1} \bar{\boldsymbol{\Sigma}} \boldsymbol{C}_{t+1}^\top)^{-1}, \tag{3}$$

$$\boldsymbol{\mu}_{t+1} = \bar{\boldsymbol{\mu}} + \boldsymbol{K} \tilde{\boldsymbol{y}}_{t+1}, \tag{4}$$

$$\boldsymbol{\Sigma}_{t+1} = (\boldsymbol{I} - \boldsymbol{K} \boldsymbol{C}_{t+1}) \bar{\boldsymbol{\Sigma}}, \tag{5}$$

where $\boldsymbol{A}_t = \left.\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}}\right|_{\boldsymbol{x}=\boldsymbol{\mu}_t, \boldsymbol{u}=\boldsymbol{u}_t}$ is the Jacobian of the dynamics w/r/t the state, evaluated at the prior mean $\boldsymbol{\mu}_t$, and $\boldsymbol{C}_{t+1} = \left.\frac{\partial \boldsymbol{h}}{\partial \boldsymbol{x}}\right|_{\boldsymbol{x}=\bar{\boldsymbol{\mu}}}$ is the measurement Jacobian, evaluated at the *predicted mean* $\bar{\boldsymbol{\mu}}$, and $\tilde{\boldsymbol{y}}_{t+1} = \boldsymbol{y}_{t+1} - \boldsymbol{h}(\bar{\boldsymbol{\mu}})$ is the "measurement innovation."

In this problem, we will show that these update equations are a single step of the Gauss-Newton method applied to the MAP problem:

$$\max_{\boldsymbol{x}_t, \boldsymbol{x}_{t+1}} \quad p(\boldsymbol{x}_t, \boldsymbol{x}_{t+1} \mid \boldsymbol{u}_t, \boldsymbol{y}_{t+1}),$$

where the prior $p(\boldsymbol{x}_t) = \mathcal{N}(\boldsymbol{\mu}_t, \boldsymbol{\Sigma}_t)$.

(a) ✎ (1 pts) Using Bayes' rule, argue the optimization problem above is equivalent to (i.e., has the same optimizer as) the problem:

$$\min_{\boldsymbol{x}_t, \boldsymbol{x}_{t+1}} \quad -\log p(\boldsymbol{x}_t) - \log p(\boldsymbol{x}_{t+1} \mid \boldsymbol{x}_t, \boldsymbol{u}_t) - \log p(\boldsymbol{y}_{t+1} \mid \boldsymbol{x}_{t+1}).$$

(b) ✎ (2 pts) Show this optimization can be written as a nonlinear least squares problem,

$$\min_{\boldsymbol{x}_t, \boldsymbol{x}_{t+1}} \quad \frac{1}{2} \boldsymbol{r}_p(\boldsymbol{x}_t)^\top \boldsymbol{r}_p(\boldsymbol{x}_t) + \frac{1}{2} \boldsymbol{r}_d(\boldsymbol{x}_t, \boldsymbol{x}_{t+1})^\top \boldsymbol{r}_d(\boldsymbol{x}_t, \boldsymbol{x}_{t+1}) + \frac{1}{2} \boldsymbol{r}_m(\boldsymbol{x}_{t+1})^\top \boldsymbol{r}_m(\boldsymbol{x}_{t+1}),$$

where $\boldsymbol{r}_p(\boldsymbol{x}_t) = \boldsymbol{\Sigma}_t^{-1/2}(\boldsymbol{x}_t - \boldsymbol{\mu}_t)$ is a prior residual, $\boldsymbol{r}_d(\boldsymbol{x}_t, \boldsymbol{x}_{t+1}) = \boldsymbol{Q}^{-1/2}(\boldsymbol{x}_{t+1} - \boldsymbol{f}(\boldsymbol{x}_t, \boldsymbol{u}_t))$ is a dynamics residual, and $\boldsymbol{r}_m(\boldsymbol{x}_{t+1}) = \boldsymbol{R}^{-1/2}(\boldsymbol{y}_{t+1} - \boldsymbol{h}(\boldsymbol{x}_{t+1}))$ is a measurement residual.

(c) ✎ (1 pts) Let $\boldsymbol{x} = \begin{bmatrix} \boldsymbol{x}_t^\top & \boldsymbol{x}_{t+1}^\top \end{bmatrix}^\top \in \mathbb{R}^{2n}$. Compute the Jacobians $\boldsymbol{J}_p = \frac{\partial \boldsymbol{r}_p}{\partial \boldsymbol{x}}$, $\boldsymbol{J}_d = \frac{\partial \boldsymbol{r}_d}{\partial \boldsymbol{x}}$, and

$\boldsymbol{J}_m = \frac{\partial \boldsymbol{r}_m}{\partial \boldsymbol{x}}$ as block row matrices.

(d) ✎ (2 pts) Applying Gauss-Newton to this problem requires solving the normal equations:

$$(\boldsymbol{J}_p^\top \boldsymbol{J}_p + \boldsymbol{J}_d^\top \boldsymbol{J}_d + \boldsymbol{J}_m^\top \boldsymbol{J}_m)\, \delta \boldsymbol{x} = -\boldsymbol{J}_p^\top \boldsymbol{r}_p - \boldsymbol{J}_d^\top \boldsymbol{r}_d - \boldsymbol{J}_m^\top \boldsymbol{r}_m,$$

where all Jacobians and residuals are evaluated at a current guess $\boldsymbol{x}_t^{(k)}, \boldsymbol{x}_{t+1}^{(k)}$.

Choose the initial guess $\boldsymbol{x}_t^{(0)} = \boldsymbol{\mu}_t,\ \boldsymbol{x}_{t+1}^{(0)} = \boldsymbol{f}(\boldsymbol{\mu}_t, \boldsymbol{u}_t) = \bar{\boldsymbol{\mu}}$. Using your Jacobians from part (c), show this yields normal equations of the form:

$$\begin{bmatrix} \boldsymbol{\Sigma}_t^{-1} + \boldsymbol{A}_t^\top \boldsymbol{Q}^{-1} \boldsymbol{A}_t & -\boldsymbol{A}_t^\top \boldsymbol{Q}^{-1} \\ -\boldsymbol{Q}^{-1} \boldsymbol{A}_t & \boldsymbol{Q}^{-1} + \boldsymbol{C}_{t+1}^\top \boldsymbol{R}^{-1} \boldsymbol{C}_{t+1} \end{bmatrix} \begin{bmatrix} \delta \boldsymbol{x}_t \\ \delta \boldsymbol{x}_{t+1} \end{bmatrix} = \begin{bmatrix} \boldsymbol{0} \\ \boldsymbol{C}_{t+1}^\top \boldsymbol{R}^{-1} \tilde{\boldsymbol{y}}_{t+1} \end{bmatrix}, \tag{6}$$

where $\boldsymbol{A}_t, \boldsymbol{C}_{t+1}$ are the same Jacobians (evaluated at the same points) as in the EKF equations (1)--(5).

(e) ✎ (4 pts) Eliminate $\delta \boldsymbol{x}_t$ from the normal equations (6) and show that

$$\delta \boldsymbol{x}_{t+1} = \boldsymbol{K} \tilde{\boldsymbol{y}}_{t+1},$$

where $\boldsymbol{K}$ is the Kalman gain from (3). Since our initial guess was $\boldsymbol{x}_{t+1}^{(0)} = \bar{\boldsymbol{\mu}}$, the Gauss-Newton update gives $\boldsymbol{x}_{t+1}^{(1)} = \bar{\boldsymbol{\mu}} + \boldsymbol{K} \tilde{\boldsymbol{y}}_{t+1} = \boldsymbol{\mu}_{t+1}$, exactly matching the EKF mean update (4).

Hint: Solve the top line to express $\delta \boldsymbol{x}_t$ in terms of $\delta \boldsymbol{x}_{t+1}$, substitute into the bottom line, and use the Woodbury identity to show the reduced system is

$$(\bar{\boldsymbol{\Sigma}}^{-1} + \boldsymbol{C}_{t+1}^\top \boldsymbol{R}^{-1} \boldsymbol{C}_{t+1})\, \delta \boldsymbol{x}_{t+1} = \boldsymbol{C}_{t+1}^\top \boldsymbol{R}^{-1} \tilde{\boldsymbol{y}}_{t+1}.$$

Then apply Woodbury again to solve for $\delta \boldsymbol{x}_{t+1}$.

Recall: for $\boldsymbol{M} \in \mathbb{R}^{n \times n}, \boldsymbol{N} \in \mathbb{R}^{m \times m}$ invertible, $\boldsymbol{U} \in \mathbb{R}^{n \times m}, \boldsymbol{V} \in \mathbb{R}^{m \times n}$:

$$(\boldsymbol{M} + \boldsymbol{U}\boldsymbol{N}\boldsymbol{V})^{-1} = \boldsymbol{M}^{-1} - \boldsymbol{M}^{-1}\boldsymbol{U}(\boldsymbol{N}^{-1} + \boldsymbol{V}\boldsymbol{M}^{-1}\boldsymbol{U})^{-1}\boldsymbol{V}\boldsymbol{M}^{-1}.$$

(f) ✎ (3 pts) One can also show (using the Laplace approximation we discussed in class) that the EKF posterior covariance $\boldsymbol{\Sigma}_{t+1}$ can be recovered from the Gauss-Newton Hessian approximation $\boldsymbol{J}^\top \boldsymbol{J}$. Thus, we can interpret the EKF as applying a single step of Gauss-Newton to the one-step MAP problem, extracting an approximate Gaussian posterior via the Laplace approximation, and reusing it as a prior for the next timestep.

Compare this to solving the full batch MAP problem over the horizon $0 : T$ using Gauss-Newton to convergence. Discuss computational tradeoffs, and explain why a poor linearization at one timestep is more damaging in the EKF than in the batch setting. Finally, briefly describe how a sliding-window smoother of window length W interpolates between the two approaches.

Solution:

4.3 Warehouse SLAM (20 pts)

A drone inspects inventory in a warehouse using an IMU (for odometry) and cameras that detect AprilTags on shelves. We jointly estimate T drone poses $\boldsymbol{T}_t \in SE(3)$ and M landmark positions $\boldsymbol{p}_\ell \in \mathbb{R}^3$ by solving a nonlinear least-squares problem over a factor graph. Throughout, Log and Exp denote the $SE(3)$ logarithmic and exponential maps (`jaxlie.SE3.log()` and `jaxlie.SE3.exp()`), and $\boldsymbol{T}^{-1}\boldsymbol{p}$ denotes the $SE(3)$ action transforming $\boldsymbol{p}$ into the body frame.

The graph has three factor types:

$$r_{\text{odom}} = \boldsymbol{\Sigma}_{\text{odom}}^{-1/2} \operatorname{Log}\big((\boldsymbol{T}_{ij}^{\text{meas}})^{-1} \cdot \boldsymbol{T}_i^{-1} \cdot \boldsymbol{T}_j\big) \in \mathbb{R}^6 \qquad \text{(odometry)} \tag{7}$$

$$r_{\text{lm}} = \boldsymbol{\Sigma}_{\text{obs}}^{-1/2}\Big(\boldsymbol{T}_t^{-1}\,\boldsymbol{p}_\ell - \boldsymbol{z}_t^\ell\Big) \in \mathbb{R}^3 \qquad \text{(landmark observation)} \tag{8}$$

$$r_{\text{lm_prior}} = \boldsymbol{\Sigma}_{\text{lm}}^{-1/2}\Big(\boldsymbol{p}_\ell - \boldsymbol{p}_\ell^{\text{prior}}\Big) \in \mathbb{R}^3 \qquad \text{(landmark prior)} \tag{9}$$

The objective is $\min \frac{1}{2}\sum_k \|r_k\|^2$. We solve with Gauss-Newton, using the retraction $\boldsymbol{T}_t^+ = \boldsymbol{T}_t \cdot \operatorname{Exp}(\delta_t)$ to stay on $SE(3)$.

We provide utilities for dataloading and visualization in `run_slam.py`. You implement the functions below in `solutions.py`.

(a) (2 pts) Implement `state_add(state, delta)`, which applies a tangent-space update $\delta \in \mathbb{R}^{6T+3M}$ to a `SlamState`. The first $6T$ entries update poses via retraction; the remaining $3M$ update landmarks by vector addition.

(b) (2 pts) Implement `residual_odometry(T_i, T_j, T_ij_meas, sqrt_info)` returning r_{odom} as in (7).

(c) (1 pts) Implement `residual_landmark(T_i, p_l, z_il_meas, sqrt_info)` returning r_{lm} as in (8).

(d) (1 pts) Implement `residual_lm_prior(p_l, p_l_prior, sqrt_info)` returning $r_{\text{lm_prior}}$ as in (9).

(e) (4 pts) Implement `robustify_huber(residual, delta)`, which takes a residual $r_k \in \mathbb{R}^d$ and returns a modified residual $\tilde{r}_k$ such that $\frac{1}{2}\|\tilde{r}_k\|^2$ equals the Huber cost:

$$\rho_\delta(s) = \begin{cases} \frac{1}{2}s^2 & s \le \delta \\ \delta\, s - \frac{1}{2}\delta^2 & s > \delta \end{cases}, \qquad \text{where } s = \|r_k\|.$$

Hint: find a scalar $\alpha(s)$ such that $\frac{1}{2}\|\alpha\, r_k\|^2 = \rho_\delta(s)$, and return $\alpha\, r_k$. Because Gauss-Newton linearizes $\tilde{r}_k$, the weight α should be treated as constant w.r.t. δ (i.e., use `jax.lax.stop_gradient` on α).

(f) (3 pts) Implement `gauss_newton_step(state, f, regularization=1e-6)`. Evaluate the stacked residual at the current state, form the Gauss-Newton system via `jax.jacfwd`, solve for the update δ^* (using the `regularization` parameter for numerical conditioning), and retract. Return the updated state and the cost $\frac{1}{2}\|r\|^2$.

(g) (1 pt) Now we can use our SLAM solver to estimate the drone's trajectory. Run:

- `python run_slam.py --dataset data/dataset_clean.npz --huber-delta None`
- `python run_slam.py --dataset data/dataset_outliers.npz --huber-delta None`

The second dataset has 1% of landmark observations corrupted with random outliers. Include both plots and explain why Gauss-Newton fails so drastically in the outlier case.

(h) (1 pt) Try adding a Huber loss: run `python run_slam.py --dataset data/dataset_outliers.npz --huber-delta [HUBER]` for several values of δ. Report your best δ, and include its plot in your writeup. In a few words/equations, describe how the problem changes as $\delta \to 0$.

(i) (1 pt) Why is the landmark prior (9) necessary? What would happen if we removed it?

(j) (2 pt) Even with accurate trajectory estimates, the drone path may drift inside shelves (which is non-physical). How could we modify our SLAM problem / solver if we wanted to guarantee this didn't happen (assuming the shelf locations are known in the world frame)?

(k) (2 pts) Run `python run_slam.py --spy-hessian` to generate a spy plot of $\boldsymbol{J}^\top \boldsymbol{J}$. Include it in your writeup. You should see roughly four blocks; explain what factors each block corresponds to. Our implementation uses dense linear algebra: what is the dimension of the linear system, how does the cost scale with T and M?

Besides exploiting sparsity, what other ways could we ensure that our SLAM problem would remain tractable over very long horizons?

Solution:

4.4 AI Usage Survey

✎ (2 pts) Please describe how you used AI (if at all) while completing this homework. Note that you will not be penalized for AI usage, regardless of how extensively you used it, as long as you acknowledge it. We are generally curious about how we can improve the course material and better help with your learning!

References

[1] Taylor Howell, Nimrod Gileadi, Saran Tunyasuvunakool, Kevin Zakka, Tom Erez, and Yuval Tassa. Predictive Sampling: Real-time Behaviour Synthesis with MuJoCo. dec 2022.

[2] Haoru Xue, Chaoyi Pan, Zeji Yi, Guannan Qu, and Guanya Shi. Full-order sampling-based mpc for torque-level locomotion control via diffusion-style annealing, 2024.